

MAE 6291

Internet of Things for Engineers

Prof. Kartik Bulusu, MAE Dept.

Week 11 [04/09/2026]

- Localhost
- Introduction to Docker: Images and Containers
- Guest lecture: IoT and Silicon Security: Dissecting a Real-Life IoT Attack by Chowdary Yanamadala, Technology Strategist, ARM
- Setting up a Docker on the Raspberry Pi 4B
- [Graded] In-class lab: Basic containerization process using Docker and the Raspberry Pi 4B

git clone https://github.com/gwu-mae6291-iot/spring2026_codes.git



School of Engineering
& Applied Science

Spring 2026

THE GEORGE WASHINGTON UNIVERSITY

Photo: Kartik Bulusu

Content Attribution & Sources

This course material incorporates both original content and supplementary visual aids:

- **Images with references:** Properly cited in slide captions, figure legends, or in-text attributions
- **Unreferenced images:** Sourced from Perplexity.io and used for illustrative purposes in an educational context
- **Fair use:** All materials are used in compliance with educational fair use guidelines



Final project proposals are all approved



localhost



What is an IP address ?

An **IP address** is a unique number that identifies a device on the internet or a local network.

- A **home address** but for computers, helping them find and communicate with each other.
- A **numerical label** assigned to the devices that are connected to the networks, based on the Internet Protocol.

IP addresses come in two versions:

- **IPv4** consists of a string of four 32-bit characters separated by a dot, like **192.168.0.1**; a series of four numbers, ranging from 0 (except the first one) to 255, each separated from the next by a period
- **IPv6** addresses are represented as eight groups of four hexadecimal digits, with the groups separated by colons. A typical IPv6 address might look like this: **2620:0aba2:0d01:2042:0100:8c4d:d370:72b4**.

Sources:
Medium: <https://medium.com/devgorilla/exploring-localhost-and-127-0-0-1-what-is-the-difference-c109f9ce7f29>
localhost: <https://todaybestreports.com/127-0-0-1-162893-localhost-port-usage-explain/>
IP address :<https://www.geeksforgeeks.org/what-is-local-host/>
<https://www.geeksforgeeks.org/open-systems-interconnection-model-osi/>
<https://www.avast.com/c-what-is-an-ip-address#:~:text=An%20IP%20address%20has%20two,number%20is%20the%20host%20ID.>

Types of IP Addresses

Local / Private
- automatically generated

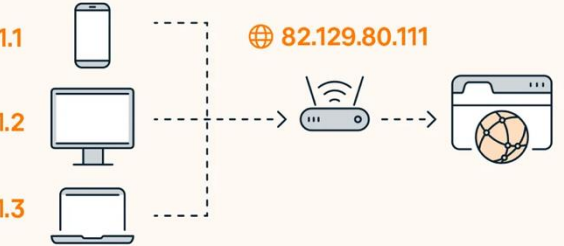
🔒 192.168.1.1

🔒 192.168.1.2

🔒 192.168.1.3

Public
- assigned by ISP

🌐 82.129.80.111



Static



- permanent
- used by servers or other important equipment

Dynamic



- occasionally changes
- used for consumer equipment

IPv4

192.168.5.18

- numeric dot-decimal notation

4.3 billion addresses
- addresses must be reused and masked

IPv6

50b2:6400:0000:0000:

6c3a:b17d:0000:10a9

- alphanumeric hexadecimal notation

7.9x10²⁸ addresses
- every device can have a unique address



IP address of your RPi connected to the GWdevice network using WiFi

ifconfig -a

```
pi@raspberrypi017: ~  
File Edit Tabs Help  
pi@raspberrypi017:~$ ifconfig -a  
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500  
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255  
    ether ea:e0:b9:19:85:8f txqueuelen 0 (Ethernet)  
    RX packets 0 bytes 0 (0.0 B)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 0 bytes 0 (0.0 B)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 169.254.247.201 netmask 255.255.0.0 broadcast 169.254.255.255  
    inet6 fe80::21e4:d41c:791a:a9d7 prefixlen 64 scopeid 0x20<link>  
    ether dc:a6:32:98:38:24 txqueuelen 1000 (Ethernet)  
    RX packets 1874 bytes 166519 (162.6 KiB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 1723 bytes 1258105 (1.1 MiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 53 bytes 5297 (5.1 KiB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 53 bytes 5297 (5.1 KiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 10.198.24.23 netmask 255.255.252.0 broadcast 10.198.27.255  
    inet6 fe80::3d8a:ccb:6a3a:2f0 prefixlen 64 scopeid 0x20<link>  
    ether dc:a6:32:98:38:25 txqueuelen 1000 (Ethernet)  
    RX packets 12 bytes 1379 (1.3 KiB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 67 bytes 8066 (7.8 KiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
pi@raspberrypi017:~$
```



Understanding 127.0.0.1: The localhost Address

The IP address [127.0.0.1](https://www.iana.org/assignments/iana-ipv4-special-assignments/) is commonly known as **localhost**.

- It is a special IP address used by a computer to refer to itself.
- Instead of connecting to an external network or the internet, this address is used for loopback communication within the same machine.

Network Configuration:

A server or program that you are running locally will frequently give you instructions on how to access it via [localhost](https://www.iana.org/assignments/iana-ipv4-special-assignments/), along with the necessary ports and extra paths.

Sources:
Medium: <https://medium.com/devgorilla/exploring-localhost-and-127-0-0-1-what-is-the-difference-c109f9ce7f29>
localhost: <https://todaybestreports.com/127-0-0-1-162893-localhost-port-usage-explain/>
IP address: <https://www.geeksforgeeks.org/what-is-local-host/>

```
bulusu — ping localhost — 80x24
(base) bulusu@SEAS-RLLJDGPPWQ ~ % ping localhost
PING localhost (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.158 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.273 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.080 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.260 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.262 ms
64 bytes from 127.0.0.1: icmp_seq=5 ttl=64 time=0.243 ms
64 bytes from 127.0.0.1: icmp_seq=6 ttl=64 time=0.247 ms
```

On your web browser:

<http://localhost>

or

<http://127.0.0.1>

When you access [127.0.0.1](https://www.iana.org/assignments/iana-ipv4-special-assignments/), you're telling your machine to communicate internally rather than looking for an external IP address.



Why do we need a localhost?

Purpose:

The primary purpose of **localhost** is to enable developers

- **To run services locally** for testing without exposing the application to the wider network and
- **To facilitate debugging**, testing web applications, and running local servers.

Development Servers



Flask
web development,
one drop at a time



Database Connections



Universality:

127.0.0.1 is configured as the **localhost** address across all systems.

Loopback Address:

The **127.0.0.1** address belongs to the **loopback network**, a reserved IP range used for network diagnostics and testing purposes.

- It allows the machine to send and receive data packets to itself without going through the network interface.

API testing

Browser Debugging

Application Logs

Sources:

Medium: <https://medium.com/devgorilla/exploring-localhost-and-127-0-0-1-what-is-the-difference-c109f9ce7f29>

localhost: <https://todaybestreports.com/127-0-0-1-162893-localhost-port-usage-explain/>

IP address: <https://www.geeksforgeeks.org/what-is-local-host/>

Node.js: By Ryan Dahl, MIT, <https://commons.wikimedia.org/w/index.php?curid=26936716>

PostgreSQL: By Daniel Lundin - https://wiki.postgresql.org/images/a/a4/PostgreSQL_logo.3colors.svg

By Vectorised from <https://labs.mysql.com/common/logos/mysql-logo.svg>, Fair use, <https://en.wikipedia.org/w/index.php?curid=67634535>

By Jamie Dihiansan <http://weblog.rubyonrails.org/2016/1/19/new-rails-identity/2> - <http://rubyonrails.org/>, CC0, <https://commons.wikimedia.org/w/index.php?curid=55052527>

<https://commons.wikimedia.org/w/index.php?curid=55052527>

By The logo is from the following website: www.mongodb.com/brand-resources, Fair use, <https://en.wikipedia.org/w/index.php?curid=74919610>

By Unknown author - <https://www.docker.com/company/newsroom/media-resources/>, GPL, <https://commons.wikimedia.org/w/index.php?curid=175844098>

<https://commons.wikimedia.org/w/index.php?curid=175844098>



What is a port in network communication ?

Definition:

A **port** is a communication endpoint used by software applications to send and receive data over a network.

- Ports serve as endpoints for communication, allowing multiple network applications to coexist on a single device.
- Ports are identified by numbers, ranging from 0 to 65535.

Usage:

Different applications use different ports.

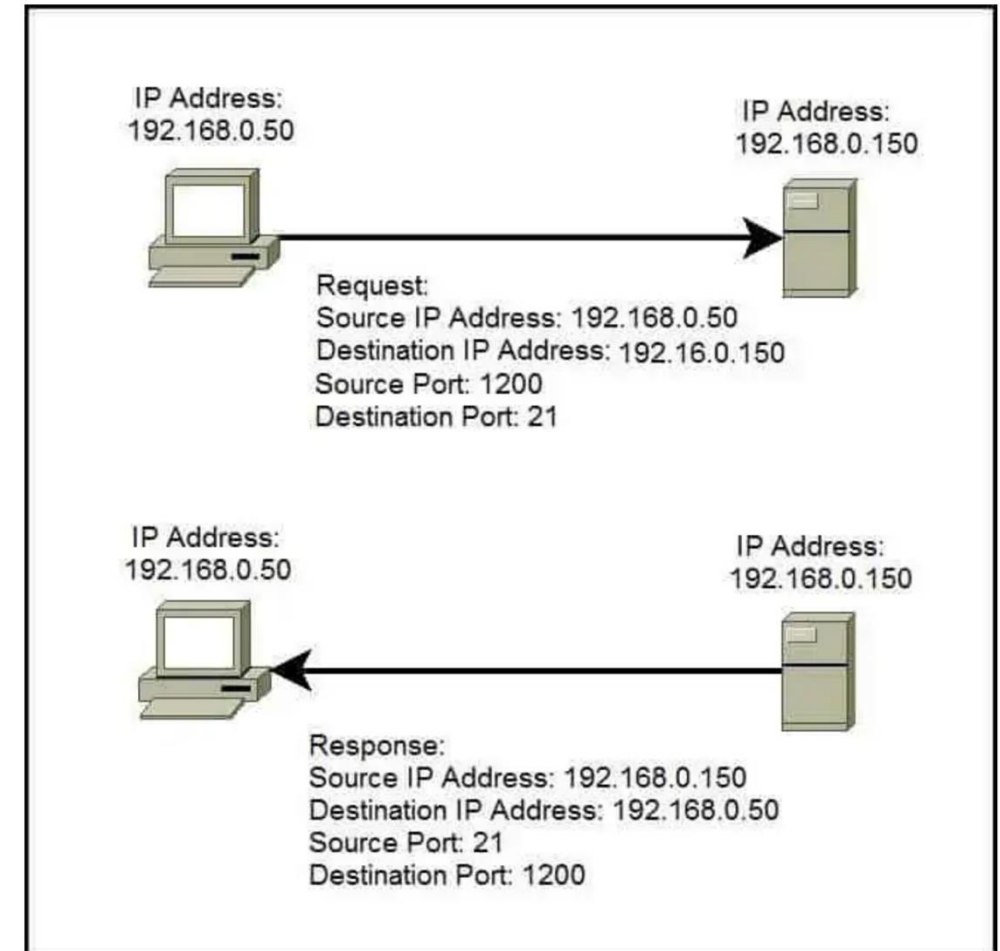
- For example, web servers typically use **port 80** for **HTTP** traffic and **port 443** for **HTTPS**.
- In the case of **127.0.0.1:62893**, the number **62893** is likely dynamically assigned by the operating system for a temporary service or local development task.

Sources:

Medium: <https://medium.com/devgorilla/exploring-localhost-and-127-0-0-1-what-is-the-difference-c109f9ce7f29>

localhost: <https://todaybestreports.com/127-0-0-1-62893-localhost-port-usage-explain/>

IP address: <https://www.geeksforgeeks.org/what-is-local-host/>
<https://study-ccna.com/ports-explained/>







Docker is an open platform for developing, shipping, and running applications, and it uses containers as the unit for distributing and testing software across environments.

- Allows us to package our code
- Allows code to be run on any computer with OS
- Simplifies the process of sharing code

TheirStack Data Pricing Resources Log in Sign up for free

Technologies > Platform And Storage > Containers And Microservices > Companies that use Docker

Companies that use Docker

The Docker Platform is the industry-leading container platform for continuous, high-velocity innovation, enabling organizations to seamlessly build and share any application — from legacy to what comes next — and securely run them anywhere

Containers And Microservices docker.io

227,687 companies

See all Get alerted

List of companies using Docker

Technology is any of Docker API Export

Company	Country	Industry	Employees	Revenue	Technologies
Capital One	United States	Financial Services	61k		Docker
Capgemini	France	IT Services and IT Consulting	420k	\$24B	Docker
IBM	United States	IT Services and IT Consulting	333k	\$61B	Docker
Oracle	United States	IT Services and IT Consulting	207k	\$50B	Docker

Docker customers by country (61)

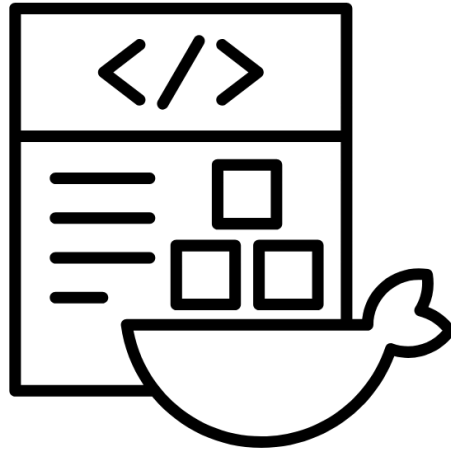
United States	30,434
United Kingdom	6,632
Germany	5,388
France	5,173
India	4,999
Canada	3,272
Spain	2,983
Netherlands	2,216
Brazil	1,832

Salary Expectations

Based on recent 2026 data, salaries for Docker-skilled professionals vary by experience:

- **Entry-Level (0-2 years):** Typically ranges from **\$59,900** to **\$100,276**.
- **Mid-Level (2-5 years):** Averages around **\$108,051**.
- **Senior/Lead (5-10+ years):** Top earners can make between **\$130,000** and **\$198,000+**.
- **Specialized High-End:** Senior roles at companies like OpenAI or SpaceX can reach total compensation of **\$250,000 to \$490,000**. Remote Rocketship +2





Docker Images

Contains:

1. Technologies or software we need
2. Runtimes
3. Tools and instructions need to run our code



Docker Containers

- Runnable instance with an image
- Multiple containers can be created using one image

- Allows us to package our code
- Allows code to be run on any computer with OS
- Simplifies the process of sharing code



Step 0: Install Docker



<https://www.docker.com>

A screenshot of the Docker website homepage. The header includes a navigation bar with links for AI, Products, Developers, Pricing, Support, Blog, and Company, along with a search icon, a "Sign In" button, and a "Get Started" button. The main heading reads "A safer container ecosystem, for everyone", followed by a subheading: "Free hardened images give every developer a trusted starting point, with enterprise options for SLAs, compliance, and extended lifecycle security." Below this are two buttons: "Get started with DHI" and "Download Docker Desktop". A dark blue modal window is open in the center, displaying download links for various operating systems: "Download for Mac - Apple Silicon", "Download for Mac - Intel Chip", "Download for Windows - AMD64", "Download for Windows - ARM64", and "Download for Linux". To the right of the modal, there is a section titled "Near-zero... now ac..." and another section titled "Docker Hardened Images Are Now FREE (Apac...)" with a video player showing the text "hardened images" and "Secure from the first pull".



Step 1: Check if you have Docker



docker --version

docker run hello-world

```
test_Docker — -zsh — 82x29

[(base) bulusu@SEAS-RLLJDGPPWQ test_Docker % docker --version ]
Docker version 29.3.1, build c2be9cc
[(base) bulusu@SEAS-RLLJDGPPWQ test_Docker % ]
[(base) bulusu@SEAS-RLLJDGPPWQ test_Docker % ]
[(base) bulusu@SEAS-RLLJDGPPWQ test_Docker % ]
[(base) bulusu@SEAS-RLLJDGPPWQ test_Docker % docker run hello-world ]

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (arm64v8)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

(base) bulusu@SEAS-RLLJDGPPWQ test_Docker %
```



Step 2: Create your Python code and a requirements.txt file

The screenshot shows the Thonny Python IDE with a file named `main.py` open. The code defines a FastAPI application with a single GET endpoint `/` that returns a JSON response. The shell window shows the execution of the code, displaying the response for the `/` endpoint.

```

1 from fastapi import FastAPI
2
3 app = FastAPI()
4
5 @app.get("/")
6 def read_root():
7     return {"message": "Hello from Dockerized Python!"}

```

Shell output:

```

Python 3.10.11 (/Applications/Thonny.app/Contents/Frameworks/Python.framework/Versions/3.10/bin/python3.10)
>>> | app is an instance of FastAPI.
      .get("/") is a method on app that returns
      a decorator function.

      Applying that decorator to read_root tells FastAPI:
      "For an HTTP GET request to /, call read_root() and
      use its return value as the response."

```

The screenshot shows a file named `requirements.txt` containing the following dependencies:

```

fastapi[standard]
uvloop==0.20.0
httptools==0.6.1
uvicorn[standard]

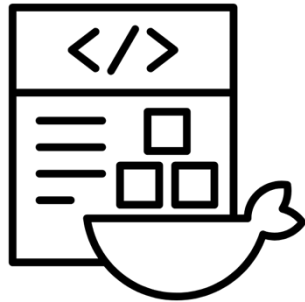
```

FastAPI is a modern, fast (high-performance), web framework for building APIs with Python based on standard Python type hints.

Uvicorn is a lightning-fast, lightweight [ASGI web server](#) for Python, designed for high-performance asynchronous frameworks like **FastAPI** and **Starlette**. It acts as a bridge, accepting HTTP requests and passing them to Python applications using **uvloop** and **httptools** for speed, supporting *HTTP/1.1* and *WebSockets*.

Save these files in a folder of your choice
Folder for the In-class example: test_Docker

Step 3: Create a DockerFile



Docker Images

Contains:

1. Technologies or software we need
2. Runtimes
3. Tools and instructions need to run our code

Save the “DockerFile” in the same folder as Step 2

```

1  # syntax=docker/dockerfile:1
2
3  ## === COMMENT: Tells Docker which Dockerfile syntax/features to use (here, the v1 "modern" syntax).
4
5  ## === COMMENT: Uses the official python:3.12-slim image as the base, giving you Python 3.12 on a minimal Debian-based image.
6  FROM python:3.12-slim
7
8  ## === COMMENT: Sets an environment variable so Python does not create .pyc bytecode files, keeping the image smaller and the filesystem cleaner.
9  ## === COMMENT: Forces Python to run in unbuffered mode so logs are flushed immediately to stdout/stderr (important for seeing logs in Docker).
10 ENV PYTHONDONTWRITEBYTECODE=1
11 ENV PYTHONUNBUFFERED=1
12
13 ## === COMMENT: Set work directory
14 ## === COMMENT: Sets /app as the working directory inside the container; all following relative paths and commands run from here.
15 WORKDIR /app
16
17 ## === COMMENT: Install dependencies
18 COPY requirements.txt .
19
20 RUN apt-get update && \
21     apt-get install -y --no-install-recommends \
22         build-essential \
23         gcc \
24         libffi-dev \
25         libssl-dev \
26         python3-dev && \
27         rm -rf /var/lib/apt/lists/*
28
29 RUN pip install --upgrade pip && \
30     pip install --no-cache-dir -r requirements.txt
31
32 ## === COMMENT: Copy app code
33 ## === COMMENT: Copies all files from your project directory (on the host) into /app in the container, including your application code.
34 COPY . .
35
36 ## === COMMENT: Expose port (inside container)
37 ## === COMMENT: Documents that the container listens on port 8000; used by tooling and for clarity, but does not itself publish the port.
38 EXPOSE 8000
39
40 ## === COMMENT: Start the app
41 ## === COMMENT: Defines the default command when the container starts: runs Uvicorn to serve the ASGI app app from the main module, binding on all inte
42 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
43

```

Uses the official slim Python 3.12 image as the base

Sets an environment variable so Python does not write .pyc bytecode files to disk and unbuffers Python outputs

Sets /app as the working directory for all following commands

Copies the requirements.txt file from your host into the working directory (/app) inside the image so you can install Python dependencies

This RUN layer installs several system packages and removes cached index files to reduce the final image size

This RUN layer installs pip and upgrades it, and installed Python packages from requirements.txt

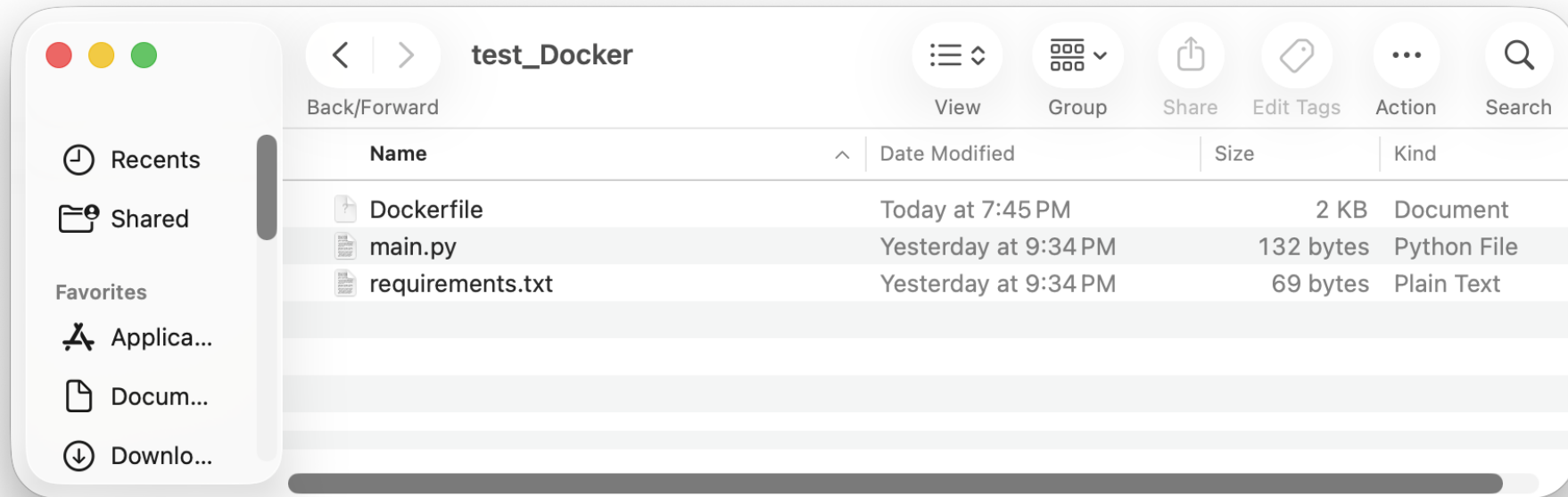
Copies everything from the current build context directory on your host into the current working directory in the image (/app)

Documents that the containerized application listens on TCP port 8000

Defines the default command that runs when a container is started from this image



What you have so far ...



docker build -t python-app:latest .

```
test_Docker — -zsh — 125x29
[(base) bulusu@SEAS-RLLJDGPPWQ test_Docker % docker build -t python-app:latest .
[+] Building 5.2s (13/13) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.85kB
=> resolve image config for docker-image://docker.io/docker/dockerfile:1
=> CACHED docker-image://docker.io/docker/dockerfile:1@sha256:2780b5c3bab67f1f76c781860de469442999ed1a0d7992a
=> => resolve docker.io/docker/dockerfile:1@sha256:2780b5c3bab67f1f76c781860de469442999ed1a0d7992a5efdf2cffc0
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/6] FROM docker.io/library/python:3.12-slim@sha256:5072b08ad74609c5329ab4085a96dfa873de565fb4751a4cfd7d
=> => resolve docker.io/library/python:3.12-slim@sha256:5072b08ad74609c5329ab4085a96dfa873de565fb4751a4cfd7d
=> [internal] load build context
=> => transferring context: 8.14kB
=> CACHED [2/6] WORKDIR /app
=> CACHED [3/6] COPY requirements.txt .
=> CACHED [4/6] RUN apt-get update && apt-get install -y --no-install-recommends build-essential
=> CACHED [5/6] RUN pip install --upgrade pip && pip install --no-cache-dir -r requirements.txt
=> [6/6] COPY . .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:db9ce1f6779ce3c2a1414d6b1c33d64fcc070f5baac786306c77b3805e96ebe0
=> => exporting config sha256:9911461a3db223af27ce9a9fcf68c50aaa2ac18ec9e3f9a5fb63fa69d7ec9360
=> => exporting attestation manifest sha256:6a14fc33b2120187ca98bf98ed9c72ff3af6354656ad49ac4d42edc1db3c40bc
=> => exporting manifest list sha256:316d2f1966ae1555630ad118b9418bf6e4c5c7571babc76f0e03bf258aa7ff4
=> => naming to docker.io/library/python-app:latest
=> => unpacking to docker.io/library/python-app:latest
(base) bulusu@SEAS-RLLJDGPPWQ test_Docker %
```

python-app is the image name,
latest is the tag generated using
the option, **-t**

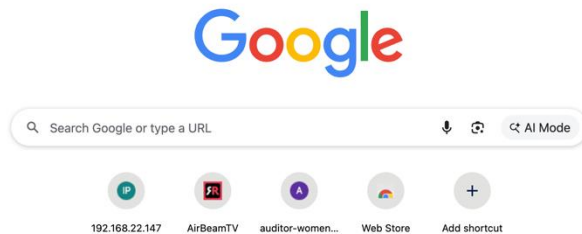
. tells Docker to use the current
directory as the build context




```
docker run -d -p 8000:8000 --name python-app-container python-app:latest
```

test_Docker — -zsh — 118x6

```
((base) bulusu@SEAS-RLLJDGPPWQ test_Docker % docker run -d -p 8000:8000 --name python-app-container python-app:latest  
49e0c1ea642329267f5dceb281081a14723729cb08b7196b0905b173c612f927  
(base) bulusu@SEAS-RLLJDGPPWQ test_Docker %
```



-d runs it in the background.

-p 8000:8000 maps host port 8000 to container port 8000 so you can reach it at <http://localhost:8000>

--name python-app-container gives the container a friendly name.



Try these and get a graded

```
docker ps
```

```
docker stop python-app-container
```

```
docker rm python-app-container
```

```
docker images
```

```
docker image rm python-app:latest
```

```
docker build -t python-app:latest .
```

```
docker run -d -p YourIP:8000:8000 --name python-app-container python-app:latest
```



Someone should summarize what we learned today

